

A Programmatic Introduction to SCTP



Since we've acquainted ourselves with the features and advantages of SCTP last month, let us put the concepts to practice by running through a simple client-server program using C.

*S*CTP socket APIs have been designed to maintain consistency with existing socket APIs. This helps in migrating existing TCP/UDP applications with the least effort. Let us have a look at some of them.



Note: SCTP provides two different API styles: one-to-one, and one-to-many.

Currently, we are looking only at the one-to-one scenario since it enables existing TCP applications to migrate to SCTP with minimal changes.

```
int socket(PF_INET, SOCK_STREAM, IPPROTO_SCTP);
```

Once an endpoint has been created using the socket call, it has to be associated with a local address. This is achieved by invoking the `bind()` method:

```
bind(int sd, struct sockaddr *addr, socklen_t addrlen);
```

Once a transport address has been associated with the socket, the listen method must be invoked (by the server application, i.e., one that is expecting inbound requests for new

connections), to prepare the SCTP endpoint to accept inbound associations. The syntax for the `listen` call remains the same:

```
int listen(int sd, int backlog);
```

`backlog` represents the maximum number of outstanding associations allowed in the `accept` queue of the socket `sd`.

Server applications use the `accept` method to accept inbound client connection requests. The syntax of `accept` is:

```
client_sd = accept(int sd, struct sockaddr *addr, socklen_t *addrlen);
```

And, client applications use `connect` to initiate an association establishment sequence:

```
int connect(int sd, const struct sockaddr *addr, socklen_t addrlen);
```

Once a new association has been established between peers, most of the normal send/receive socket APIs can be used for communication. For example, if we consider `sendmsg/recvmmsg` APIs: